Parcial 3 — Seguridad de archivos en la capa de kernel y `mmap()`

Universidad EAFIT · Sistemas Operativos · 2026-2 Estudiante: Juan Fernando Romero · **Modalidad:** individual
Solución: `mmap_guard`, un módulo de kernel (LKM) que vigila `mmap()` y `mprotect()` sobre un archivo protegido.
Probado en: Ubuntu 24.04.4 LTS, kernel `6.8.0-134-generic` (aarch64), gcc 13.3, en una VM. El código es portable a x86_64 (no usa registros ni instrucciones de una arquitectura concreta).

Resumen

Cuando un archivo se proyecta con `mmap()`, el kernel solo interviene dos veces: al **crear el mapeo** (se crea un `vm_area_struct`) y en el **primer fallo de página** de cada página. Después, la CPU y la MMU leen y escriben la memoria directamente, sin ninguna llamada al sistema. Por eso una defensa en espacio de usuario (`LD_PRELOAD`, envoltorios) no sirve: el atacante simplemente hace la syscall directa o usa el puntero.

`mmap_guard` se ubica en el último punto donde el kernel todavía decide: las funciones del framework LSM `security_mmap_file()` y `security_file_mprotect()`. El módulo las envuelve con **kretprobes**: lee los argumentos (archivo, permisos y tipo de mapeo) y, si la operación viola la política, cambia el valor de retorno a `-EACCES`. Así bloquea el mapeo de verdad, sin recompilar el kernel. Las reglas son:

Regla	Qué niega	Por qué
R1	Que un usuario distinto de `uid_autorizado` proyecte el archivo.	Si otro proceso no comparte sus marcos físicos, no ve cambios en vivo ni puede montar canales laterales de caché (Flush+Reload).
R2	`PROT_EXEC` sobre el archivo.	POSIX permite mapear como código un archivo **sin** bit `x`; con la regla, sus PTE siempre tienen NX y el hardware impide ejecutar código inyectado ahí.
R3	`PROT_WRITE` con `MAP_SHARED`.	Esas escrituras llegan al disco sin pasar por ningún punto de inspección (no hay `write()`).
R4	`mprotect()` que suba un mapeo existente a ejecutable o a escritura compartida.	Sin R4, R2 y R3 se saltan en dos pasos: primero se mapea con permisos inocentes y luego se suben.

Cada decisión queda auditada en `/proc/mmap_guard`, con contadores exactos, y en `dmesg`. El script `test_parcial3.sh` corre 9 ataques sin el módulo y con él: **9/9 casos correctos con el módulo, y el archivo nunca cambia en disco** (verificado con `md5sum`).

PARTE I — Las 10 preguntas diagnósticas

1. Mapeo virtual y fallos de página: de `mmap()` al primer fallo

a) La llamada `mmap()` (crea la VMA, pero no carga datos):

1. El proceso ejecuta la instrucción de syscall (`syscall` en x86_64, `svc #0` en arm64). La CPU pasa de Ring 3 a Ring 0 (EL0→EL1 en ARM) y entra a `ksys_mmap_pgoff()`, que obtiene el `struct file` del descriptor con `fget(fd)`.
2. `vm_mmap_pgoff()` (mm/util.c) llama primero a **`security_mmap_file(file, prot, flags)`**, el gancho LSM donde AppArmor/SELinux, y en este trabajo `mmap_guard`, pueden negar la operación. Después toma el candado de escritura del espacio de direcciones (`mmap_write_lock(mm)`).
3. `do_mmap()` (mm/mmap.c) valida la longitud y el desplazamiento, busca un hueco libre con `get_unmapped_area()` y traduce `prot`/`flags` a `vm_flags` (`VM_READ`, `VM_WRITE`, `VM_EXEC`, `VM_MAYREAD`,

`VM_MAYWRITE`, `VM_MAYEXEC`, `VM_SHARED`, `VM_MAYSHARE`). Aquí se aplican los permisos del descriptor: `MAP_SHARED` + `PROT_WRITE` exige `FMODE_WRITE`, y en un montaje `noexec` se niega `PROT_EXEC`.

4. `mmap_region()` crea (o fusiona) el **`struct vm_area_struct`** con `vm_start`, `vm_end`, `vm_pgoff`, `vm_flags`, `vm_page_prot` y `vm_file` (con referencia `get_file()`). Luego llama al `->mmap` del sistema de archivos (en ext4, `ext4_file_mmap()`), que instala `vma->vm_ops` (`filemap_fault`, `filemap_map_pages`, `ext4_page_mkwrite`). La VMA se inserta en el *maple tree* de `mm_struct` (desde 6.1; antes era un árbol rojo-negro).
5. **No se crea ninguna PTE ni se lee el disco** (asignación perezosa), salvo con `MAP_POPULATE`. Se libera el candado y la syscall retorna la dirección virtual.

b) El primer acceso (fallo de página):

6. El proceso hace `*ptr`. La MMU recorre la tabla de páginas desde CR3/TTBR0 (PGD → P4D → PUD → PMD → PTE) y encuentra la PTE vacía (bit P=0). Entonces dispara la excepción **#PF (vector 14)**: CR2 queda con la dirección y el código de error trae los bits P, W/R, U/S e I/D. En arm64 es un *data abort* con ESR_EL1/FAR_EL1.
7. `exc_page_fault()` → `do_user_addr_fault()` buscan la VMA, primero con el candado por VMA (`lock_vma_under_rcu`, desde 6.4) y si no con `mmap_read_lock`. Luego `access_error()` compara el tipo de acceso con `vm_flags`; si se escribe en una VMA sin `VM_WRITE`, el resultado es `SIGSEGV`.
8. `handle_mm_fault()` → `__handle_mm_fault()` crea los niveles intermedios que falten (`p4d_alloc`, `pud_alloc`, `pmd_alloc`) y llama a `handle_pte_fault()` → `do_pte_missing()` → `do_fault()`:
   - **Lectura:** `do_read_fault()` intenta primero mapear páginas vecinas que ya estén en caché (*fault-around*, `filemap_map_pages`). Luego `__do_fault()` → `vma->vm_ops->fault` = **`filemap_fault()`**, que busca el folio en el **Page Cache** (`filemap_get_folio`). Si no está, lo lee del disco con `->read_folio` (fallo mayor) y espera la E/S.
   - **Escritura en `MAP_PRIVATE`:** `do_cow_fault()` reserva una página anónima, copia el contenido y la mapea escribible solo para este proceso.
   - **Escritura en `MAP_SHARED`:** `do_shared_fault()` llama a `->page_mkwrite` (el FS reserva bloques) y marca el folio como sucio.
9. `finish_fault()` → `set_pte_at()` escribe la PTE con `vm_page_prot` (P=1, U/S=1, R/W según el caso, NX si no hay `VM_EXEC`), registra el mapeo inverso (*rmap*) y suelta el candado de la tabla (PTL). Pasar de "no presente" a "presente" no exige invalidar la TLB.
10. El kernel retorna (`iret`/`eret`), la instrucción se **reejecuta** y esta vez la MMU encuentra la PTE. Los accesos siguientes a esa página van directo a RAM, sin Ring 0. Esto es lo que muestra la Fase 3 del infograma, y por eso la defensa debe estar en los pasos 2–4 o 7–9.

2. `MAP_SHARED` frente a `MAP_PRIVATE` en `vm_area_struct`

	`MAP_SHARED`	**`MAP_PRIVATE`**
`vm_flags`	`VM_SHARED | VM_MAYSHARE`. Si el fd **no** se abrió para escribir, `do_mmap()` borra `VM_SHARED` y `VM_MAYWRITE` (queda solo `VM_MAYSHARE`).	Sin `VM_SHARED`. **`VM_MAYWRITE` sí queda puesto aunque el fd sea `O_RDONLY`** (las escrituras van a una copia).
PTE	Apunta al folio del **Page Cache**: es el mismo marco físico para todos los procesos que mapean el archivo y para `read()`/`write()`.	Al inicio apunta también al folio del Page Cache, pero **siempre en solo lectura** (`vm_page_prot` sin W). La primera escritura provoca COW y la PTE pasa a una página anónima propia.
Persistencia	Las escrituras ensucian el folio y el *writeback* las lleva al disco.	Nunca se escriben al archivo; la copia vive en memoria anónima (y puede ir a swap).
Visibilidad cruzada	Inmediata: otro proceso ve el cambio en la siguiente instrucción.	Los cambios propios son invisibles para los demás. Las páginas **aún no copiadas** sí reflejan los cambios que otros hagan al archivo (POSIX no lo especifica).

Implicaciones de seguridad. Con `MAP_SHARED`:

- Un proceso comprometido con permiso de escritura altera el archivo de forma persistente y en vivo, sin syscalls que auditar.
- Compartir el marco físico permite canales laterales (Flush+Reload, `mincore()` sobre el Page Cache).
- Si el archivo se trunca, quien lo tiene mapeado recibe `SIGBUS` (denegación de servicio).
- Hay riesgo de **TOCTOU / double fetch**: se valida un dato y otro proceso lo cambia antes de usarlo.

Con `MAP_PRIVATE` no hay persistencia, pero quedan copias de datos sensibles en memoria anónima: pueden ir a swap o a un *core dump*, y un `fork()` las hereda. Además se mezcla contenido viejo con nuevo. `mmap_guard` usa `VM_MAYSHARE`, y no `VM_SHARED`, para reconocer un mapeo compartido, justamente porque `VM_SHARED` se borra cuando el fd es de solo lectura.

3. El ciclo de vida de una página bajo Copy-On-Write

1. **Compartida y protegida:** tras `mmap(MAP_PRIVATE)` o tras `fork()`, la página física se comparte. Su `_refcount` / `_mapcount` es mayor que 1 y todas las PTE quedan **sin bit W**, aunque la VMA tenga `VM_WRITE`.
2. **Fallo de protección:** al escribir, la MMU ve P=1 y W=0, y dispara un #PF con el bit de escritura. `handle_pte_fault()` → **`do_wp_page()`**.
3. **Reutilizar o copiar:** si la página es anónima y exclusiva del proceso (`PageAnonExclusive`, desde 5.19), solo se marca escribible. Si no, **`wp_page_copy()`** reserva una página nueva, copia los 4 KiB y reemplaza la PTE con `ptep_clear_flush()` (borra la entrada e invalida la TLB) y luego `set_pte_at()` (escribible). Después decrementa la referencia a la página original.
4. **Vida anónima:** la copia pertenece al `anon_vma` del proceso; puede ir a swap y se libera con `munmap` / `exit`. El archivo nunca cambia.

Cuándo un error en las banderas lleva a escalada de privilegios. Los ejemplos son reales:

- **`VM_MAYWRITE` mal puesto:** si el kernel dejara `VM_MAYWRITE` en un `MAP_SHARED` de un fd `O_RDONLY`, un `mprotect(PROT_WRITE)` daría escritura directa sobre el Page Cache de un archivo ajeno (por ejemplo `/etc/passwd`). Por eso `do_mmap()` lo borra.
- **Dirty COW (CVE-2016-5195):** `get_user_pages()` con `FOLL_FORCE|FOLL_WRITE` (usado al escribir en `/proc/self/mem`) rompía el COW y, para reintentar, quitaba `FOLL_WRITE`. Si en esa ventana otro hilo hacía `madvise(MADV_DONTNEED)`, la copia COW desaparecía. El reintento, ya sin `FOLL_WRITE`, mapeaba **la página original del Page Cache**, y la escritura caía en el archivo de solo lectura. Así se podían modificar binarios setuid y obtener root. La corrección (2016) añadió `FOLL_COW` y comprobó que la PTE estuviera sucia. Desde 5.19 a 6.x se reescribió con `PageAnonExclusive` y `FAULT_FLAG_UNSHARE`: GUP solo escribe en páginas anónimas exclusivas.
- **Dirty Pipe (CVE-2022-0847)** tuvo la misma consecuencia, escribir en el Page Cache de archivos de solo lectura, por una bandera mal inicializada (`PIPE_BUF_FLAG_CAN_MERGE`).

Moraleja para el diseño: el Page Cache de un archivo protegido es el activo a defender, y toda ruta que pueda obtener una PTE escribible hacia él debe controlarse en el kernel.

4. LKM con hook en la System Call Table frente a LSM (`file_mmap` / `mmap_file`)

	Hook en `sys_call_table` (LKM)	**LSM (`security_mmap_file`)**
Viabilidad hoy	**Muy baja.** `sys_call_table` y `kallsyms_lookup_name` no se exportan desde 5.7. La tabla está en memoria de solo lectura y, desde 5.3, el kernel fija el bit WP de CR0. En x86 **desde 6.9 las syscalls ya no se despachan por la tabla** sino por un `switch` (`x64_sys_call`), así que parchearla no tiene efecto.	**Alta, pero hay que compilarlo dentro del kernel.** Desde 4.x los hooks son `__ro_after_init`: un LSM no se registra desde un módulo cargable. La alternativa sin recompilar es BPF-LSM, que exige `lsm=...,bpf` al arrancar.

	Hook en `sys_call_table` (LKM)	**LSM (`security_mmap_file`)**
Qué ve	Valores crudos de la syscall: el número de fd todavía no está resuelto, lo que abre la puerta a carreras (TOCTOU).	El `struct file` ya resuelto, con sus credenciales y su inodo.
Cobertura	Hay que enganchar `mmap`, `mmap2`, las variantes compat de 32 bits y `mprotect`; es fácil olvidar una.	El kernel llama al gancho en **todas** las rutas (`vm_mmap_pgoff`, también para compat). Hay ganchos aparte para `mprotect` (`file_mprotect`) y `mmap_addr`.
Estabilidad	Técnica de *rootkit*: carreras al parchear y riesgo de pánico. Puede romper KPTI/CFI.	Interfaz oficial que se puede apilar con AppArmor/SELinux.

Nota sobre el nombre: el gancho se llamaba **`file_mmap`** en kernels 2.6/3.x; hoy es **`mmap_file`** (más `mmap_addr`). El script `verificar_ia.sh` lo comprueba en `lsm_hook_defs.h`.

Decisión de este trabajo: tomar el punto de intercepción del LSM (la función `security_mmap_file`) pero instalarlo con **kretprobes** desde un LKM. Así se tiene la semántica del LSM (archivo resuelto, todas las rutas) sin recompilar y sin tocar la tabla de syscalls. Los costos están en la sección 4.2.

5. Cifrado transparente y `mmap()`

El esquema correcto es el de **fscrypt** (ext4, f2fs) o el cifrado en línea `blk-crypto`:

1. **En disco, cifrado; en el Page Cache, claro.** Al leer, el *bio* trae bloques cifrados y, al completarse, `fscrypt_decrypt_bio()` (o el hardware de cifrado en línea) los descifra **dentro del folio del Page Cache**. `filemap_fault()` mapea ese folio, así que el proceso autorizado lee texto claro con `mmap()` sin que la aplicación se entere.
2. **Al escribir de vuelta**, el FS cifra en una **página de rebote** (`fscrypt_encrypt_pagecache_blocks`) y envía esa al disco. El Page Cache nunca queda cifrado y el disco nunca queda en claro.
3. **Quién llega al texto claro.** La memoria física está aislada por las tablas de páginas: otro proceso solo alcanza ese marco si **mapea o lee el mismo archivo**. Por eso el control se hace en tres capas:
   - (a) la clave: con políticas v2, sin `FS_IOC_ADD_ENCRYPTION_KEY` el `open()` falla con `ENOKEY`;
   - (b) DAC/LSM sobre `open`/`mmap` (la regla R1 de este trabajo);
   - (c) al quitar la clave (`FS_IOC_REMOVE_ENCRYPTION_KEY`), el kernel expulsa los inodos y su Page Cache. Con `init_on_free=1` los marcos se ponen en cero al liberarse.
4. **Fugas que quedan:**
   - las copias COW (`MAP_PRIVATE`) son memoria anónima: pueden ir a swap (hay que cifrar el swap) y a *core dumps* (`MADV_DONTDUMP`, `mlock`);
   - `ptrace` y `/proc/pid/mem` (Yama `ptrace_scope`);
   - ataques físicos: *cold boot* o DMA, que se mitigan con cifrado de memoria por hardware (AMD SME/SEV, Intel TME/TDX) e IOMMU.

   `dm-crypt` (cifrado del bloque completo) no sirve para este objetivo, porque deja el Page Cache en claro para **cualquiera** que tenga permiso sobre el archivo.

6. Páginas sucias, `msync()`/`fsync()` y evasión de filtros

Cómo garantiza el kernel la consistencia:

1. En un `MAP_SHARED` escribible, la PTE empieza **sin W** (*write-notify*, `vma_wants_writenotify`). La primera escritura provoca un fallo y el kernel llama a **`->page_mkwrite`**: el FS reserva bloques y espera si la página está en *writeback* (*stable pages*). Solo entonces pone la PTE escribible y sucia, y etiqueta el folio con `PAGECACHE_TAG_DIRTY`.

2. Los hilos de *writeback* (según `dirty_expire_centisecs`, `dirty_ratio`) o un `msync(MS_SYNC)` / `fsync()` → `vfs_fsync_range()` → `file_write_and_wait_range()` llevan el folio al disco. Antes de escribirlo, `folio_clear_dirty_for_io()` → `folio_mkclean()` recorre el *rmap* y **vuelve a proteger contra escritura todas las PTE**. Así, una escritura posterior provoca un fallo nuevo y la página no se pierde ni se escribe a medias. El journal (ext4 `data=ordered`) ordena datos y metadatos.

Cómo se evade un filtro. Las escrituras por puntero **no son llamadas al sistema**:

- No pasan por `write()`, ni por `fanotify` / `inotify` (`IN_MODIFY` no se genera por almacenamientos en un mapeo), ni por un antivirus que intercepte `write`.
- El dato llega al disco segundos después, cuando lo decide el *writeback*, y sin ningún gancho de inspección del contenido.
- Un atacante puede escribir y restaurar el dato antes del escaneo, o cambiarlo **después** de que un monitor lo validó (TOCTOU).

Mitigación de este trabajo: prohibir `PROT_WRITE` + `MAP_SHARED` sobre el archivo protegido (R3) y su variante por `mprotect` (R4). Toda modificación queda obligada a pasar por `write()`, que sí es una syscall auditable. Para archivos inmutables, `fs-verity` es el complemento natural.

7. Auditoría con eBPF, `kprobes` y `fexit` sin recompilar

Un programa eBPF lo verifica el *verifier* del kernel (no puede colgar ni corromper memoria) y se compila con JIT. Puede adjuntarse a:

- **kprobe/kretprobe** sobre `security_mmap_file`, `security_file_mprotect`, `filemap_fault` o `do_wp_page`, para ver argumentos y resultado;
- **fentry/fexit**: trampolín BPF con tipos BTF, más barato que un kprobe. `fexit/security_mmap_file` ve a la vez los argumentos y el valor de retorno;
- **tracepoints** estables: `syscalls:sys_enter_mmap`, `syscalls:sys_enter_madvise`, y `exceptions:page_fault_user` en x86;
- **BPF-LSM** (`lsm/mmap_file`), el único tipo que además puede **negar** con `-EPERM`.

El filtrado (inodo, uid) se hace dentro del kernel y los eventos salen por un *ring buffer* o se acumulan en mapas hash. Patrones anómalos que se detectan así:

- `PROT_EXEC` sobre archivos sin bit `x`;
- ráfagas de `madvise(MADV_DONTNEED)` junto a escrituras en `/proc/pid/mem` (la firma de Dirty COW);
- una tasa anómala de fallos de página por proceso sobre un inodo sensible;
- `mprotect` que agrega W o X.

Ejemplo con `bpftrace`, probado en la VM (Ubuntu 24.04, bpftrace 0.20) sin recompilar nada ni cargar módulos:

```
sudo bpftrace -e 'kprobe:security_mmap_file /arg0 != 0/ {
    @[comm, str(((struct file *)arg0)->f_path.dentry->d_name.name), arg1] = count(); }'

@[poc_mmap, confidencial.txt, 1]: 3      <- tres mapeos de lectura (PROT_READ)
@[poc_mmap, confidencial.txt, 5]: 1      <- intento de mapeo EJECUTABLE (PROT_READ|PROT_EXEC)
```

La salida completa está en `evidencias/ebpf_auditoria.txt`.

Solo hace falta `CONFIG_BPF_SYSCALL` y BTF (`/sys/kernel/btf/vmlinux`), que vienen activos en los kernels de distribución.

Límite: después de instalada la PTE, los accesos ya no generan eventos. Para muestrearlos se necesitan *soft-dirty*, `userfaultfd` en modo protección contra escritura, o desmapear periódicamente. `mmap_guard` usa

exactamente la misma técnica de kprobe, pero desde un LKM para poder **bloquear** sin el parámetro de arranque que pide BPF-LSM.

8. `O_RDONLY` + `PROT_READ|PROT_WRITE` + `MAP_PRIVATE`

Sí se permite, en cualquier configuración, y es correcto que así sea. En `do_mmap()`:

- `MAP_SHARED` con `PROT_WRITE` exige `FMODE_WRITE` en el descriptor; si falta, retorna `-EACCES`.
- `MAP_PRIVATE` solo exige `FMODE_READ`, porque **las escrituras nunca vuelven al archivo**: van a una copia COW anónima. Semánticamente es lo mismo que hacer `read()` a un búfer propio y modificarlo. Por eso `VM_MAYWRITE` queda puesto y luego también funciona `mprotect(PROT_WRITE)`.

El caso 4 de `test_parcial3.sh` lo demuestra: el proceso ve "HACKEADO" en su memoria y el `md5sum` del archivo no cambia.

Riesgos que sí implica:

1. **Alteración temporal:** el proceso trabaja sobre datos que ya no corresponden al disco. Si otro componente del mismo proceso (un intérprete, un cargador) confía en esa región, se le puede engañar.
2. **Mezcla de versiones:** las páginas no copiadas siguen reflejando el archivo, así que si otro lo cambia se ve una mezcla vieja/nueva (TOCTOU).
3. **Fuga:** las copias van a swap, a *core dumps*, al hijo tras `fork()`, y son legibles por `ptrace` del mismo uid.
4. **Amplificador de bugs del kernel:** fue exactamente la puerta de Dirty COW. Un mapeo privado de solo lectura más una falla en COW equivale a escribir el archivo.
5. **Relacionado:** `PROT_EXEC` sobre un archivo **sin bit `x`** también se permite (solo lo impide un montaje `noexec`). Es una evasión real del permiso de ejecución POSIX, y por eso existe la regla R2.

9. Carreras entre `madvise(MADV_DONTNEED)` y escrituras en páginas privadas

El kernel actual las previene con varias capas:

1. **Candado de la tabla de páginas (PTL) y revalidación.** `MADV_DONTNEED` → `zap_page_range_single()` borra las PTE bajo `pte_offset_map_lock()` y acumula las páginas en un `mmu_gather`. Estas **solo se liberan después de invalidar la TLB** en todas las CPUs (`tlb_finish_mmu`), así que ninguna CPU puede seguir escribiendo en una página ya liberada. Un fallo concurrente prepara su página fuera del candado y, al tomarlo, comprueba `pte_same()`: si la PTE cambió, descarta su trabajo y reintenta. No hay actualizaciones perdidas.
2. **Candados de VMA.** `madvise` toma `mmap_read_lock` (o el candado por VMA). Las operaciones que cambian la estructura (`munmap`, `mprotect`) toman el de escritura y no se cruzan con ella. `MADV_DONTNEED` falla sobre VMAs bloqueadas con `mlock` (`VM_LOCKED`).
3. **GUP seguro (el arreglo de Dirty COW).** Desde 5.19/6.x, `get_user_pages()` con `FOLL_FORCE|FOLL_WRITE` sobre un mapeo privado de solo lectura solo escribe si la página mapeada es **anónima y exclusiva** (`PageAnonExclusive`), y lo comprueba en `can_follow_write_pte()`. Si un `DONTNEED` la tiró, el reintento produce un **nuevo** fallo COW (`FAULT_FLAG_UNSHARE`) y jamás escribe sobre el Page Cache.
4. **Liberación diferida con RCU** de las tablas de páginas, para que un recorrido sin candado (GUP rápido) no lea una tabla ya liberada.

Evidencia: el caso 9 de `test_parcial3.sh` lanza el PoC de Dirty COW (un hilo con `MADV_DONTNEED` en bucle y otro escribiendo por `/proc/self/mem`) contra el archivo protegido en el kernel 6.8, **sin** el módulo. El archivo no cambia: el kernel resiste. Con `mmap_guard`, el atacante (caso 8) ni siquiera obtiene el mapeo.

10. `struct page`, tablas de páginas y bits de la MMU frente a la inyección de código

- **`struct page` / `folio`:** el descriptor de cada marco físico. Guarda `flags` (`PG_locked`, `PG_dirty`, `PG_anon_exclusive`…), `_refcount`, `_mapcount`, `mapping` (el `address_space` del archivo o el `anon_vma`) e

`index`. Con esto el kernel sabe quién comparte un marco y si se puede escribir sin copiar: es la base del COW y del *rmap* que se usa para revocar permisos (`folio_mkclean`).

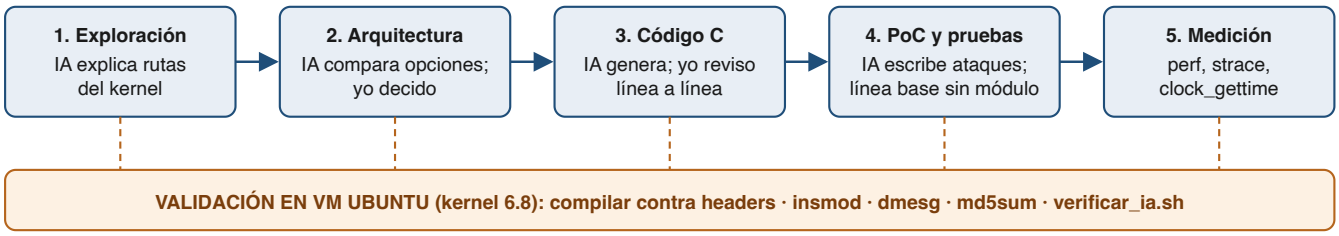
- **Tablas multinivel (PGD → P4D → PUD → PMD → PTE):** cada proceso tiene su propio árbol (CR3/TTBR0). Una dirección virtual solo llega a un marco si existe una PTE en **su** árbol, y eso es el aislamiento entre procesos. Con 4 niveles, P4D se "pliega" (`LA57` activa el 5.º nivel). KPTI usa árboles separados para usuario y kernel (mitigación de Meltdown).
- **Bits de protección de la PTE:**
  - **P** (presente): si es 0, cualquier acceso cae al kernel (#PF). Es lo que permite el control en el fallo de página.
  - **R/W**: si es 0, toda escritura provoca un fallo (COW, *write-notify*, solo lectura).
  - **U/S**: las páginas del kernel tienen U/S=0, así que el código de usuario no puede leerlas. SMEP/SMAP (PXN/PAN en arm64) impiden además que el kernel ejecute o lea memoria de usuario por accidente.
  - **NX/XD** (bit 63, con `EFER.NXE`; UXN en arm64): si está puesto, un salto a esa página provoca un #PF con el bit I/D en lugar de ejecutar. El kernel lo calcula desde `vm_flags` con `vm_get_page_prot()`: sin `VM_EXEC`, la página siempre tiene NX.

Contención de la inyección: si un atacante escribe *shellcode* en un mapeo de datos, la CPU se niega a ejecutarlo porque esa página tiene NX. El atacante entonces intenta **conseguir** `VM_EXEC`: mapear el archivo con `PROT_EXEC` o hacer `mprotect(PROT_EXEC)`. `mmap_guard` cierra ese camino (R2 y R4) para el archivo protegido. Así sus PTE **siempre** tienen NX y la defensa termina en el hardware, no en software. Complementos: *Memory Protection Keys* (PKU) y `CONFIG_STRICT_KERNEL_RWX` (W^X en el propio kernel).

PARTE II — Estrategia con apoyo de Inteligencia Artificial

Herramienta usada: Claude (Anthropic), mediante **Claude Code**, un agente de IA en la terminal que lee archivos, propone código y ejecuta comandos con mi autorización. Regla de trabajo: **la IA propone y el kernel decide**. Ninguna afirmación técnica ni línea de código se aceptó sin compilarla contra los headers del kernel real y ejecutarla en una VM Ubuntu.

3.1 Flujo de trabajo asistido por IA



Fase	Rol de la IA	Cómo validé
Exploración arquitectónica	Explicar la ruta `sys_mmap → vm_mmap_pgoff → do_mmap → mmap_region` y el manejo de fallos; listar los ganchos LSM relacionados con memoria; explicar `vm_flags` y COW.	Contrastando con los headers instalados (`lsm_hook_defs.h`, `mm.h`) y con `/proc/kallsyms`, usando `verificar_ia.sh`.
Generación y verificación de código C	Proponer el esqueleto del LKM con kretprobes, el `seq_file` de `/proc`, el Makefile de kbuild y el manejo de errores en `init`.	Compilando contra `/lib/modules/$(uname -r)/build` con cero advertencias, cargando con `insmod` y revisando `dmesg` y `/proc/mmap_guard`.

Fase	Rol de la IA	Cómo validé
Casos de prueba y PoC	Escribir `poc_mmap.c`: 7 modos de ataque, incluido el PoC controlado de Dirty COW solo contra un archivo de prueba propio. Escribir `test_parcial3.sh` (línea base sin módulo frente a con módulo) y `bench_mmap.c`.	Ejecutando cada caso primero **sin** el módulo, para confirmar que el ataque funciona (si no, la prueba no prueba nada), y luego con él; verificando con `md5sum`.

3.2 Ingeniería de prompts y verificación crítica

Prompts estructurados usados

#	Rol	Contexto	Restricción	Resultado esperado	Resultado obtenido y validación
P1	"Eres ingeniero del subsistema mm de Linux."	Ubuntu 24.04, kernel 6.8, x86_64 y arm64. Necesito interceptar `mmap()` de un archivo.	Cita solo funciones que existan en 6.8, con su archivo fuente. Marca cuáles son exportadas a módulos.	Mapa de la ruta `mmap()` → primer fallo, con los puntos donde un LKM puede intervenir sin recompilar.	Ruta confirmada contra los headers 6.8. El nombre de gancho que trae el propio enunciado, `file_mmap`, **no existe** en 6.8; el real es `mmap_file` (fila 3 de la tabla de alucinaciones).
P2	"Eres arquitecto de seguridad de kernel."	Comparar: hook de `sys_call_table`, LSM compilado, BPF-LSM, kretprobe sobre `security_mmap_file`, eBPF solo auditoría.	Tabla con viabilidad **en un kernel de distribución sin recompilar ni cambiar parámetros de arranque**.	Matriz de *trade-offs* para elegir la arquitectura.	Las vías "clásicas" (tabla de syscalls con `kallsyms_lookup_name` + `write_cr0`, LSM con `security_add_hooks`) se descartaron al verificarlas (filas 1, 2 y 6). Se eligió kretprobe.
P3	"Eres desarrollador de módulos de kernel."	Política R1–R4 sobre un archivo identificado por (dispositivo, inodo).	Portable x86_64/arm64; sin símbolos no exportados; sin escribir `vm_flags`; GPL; contadores atómicos; nada que duerma dentro del handler.	`mmap_guard.c` + Makefile kbuild que compile sin advertencias.	Compiló sin errores, pero kbuild avisó "*the compiler differs from the one used to build the kernel*" → el Makefile ahora usa el gcc de `CONFIG_CC_VERSION_TEXT` y la compilación queda **sin advertencias**. Se verificó que `regs->di` no es portable (fila 5).
P4	"Eres revisor de seguridad ofensiva."	Este es mi módulo (código pegado).	Busca fallos *fail-open*, carreras, contexto atómico y fugas de referencias. No reescribas: lista hallazgos.	Lista de debilidades del propio diseño.	(1) Si se agotan las instancias del kretprobe (`maxactive`), la llamada **no se revisa** (*fail-open*) → `maxactive=256` y `nmissed` visible en `/proc` (fila 7). (2) `VM_SHARED` se borra en un `MAP_SHARED` de fd `O_RDONLY` → se usa `VM_MAYSHARE`. (3) En la prueba, `insmod` con un archivo inexistente decía "*Unknown symbol in module*" (así traduce `-ENOENT`) → el módulo retorna `-EINVAL` ("*Invalid parameters*").

#	Rol	Contexto	Restricción	Resultado esperado	Resultado obtenido y validación
P5	"Eres investigador de vulnerabilidades."	Laboratorio propio en VM. Archivo de prueba propio en `/srv/mmap_guard`.	PoC controlado de Dirty COW que **solo** ataque ese archivo, termine en N segundos y reporte con `pread` si cambió el disco.	Modo `carrera` de `poc_mmap.c`.	La primera versión no demostraba que la carrera **sí ocurrió** → se añadieron contadores: ≈3 millones de `madvise` y ≈1,9 millones de escrituras en 2 s, y el archivo sigue íntegro en 6.8. Con el módulo, el `mmap` del atacante se niega.
P6	"Eres ingeniero de rendimiento."	Necesito el sobrecosto del módulo en `mmap()` y en fallos de página.	Usar `clock_gettime`, `getrusage`, `perf stat`, `strace -c`; 3 repeticiones; caché fría y caliente.	`bench_mmap.c` + `medir_overhead.sh`.	Se esperaban 16 384 fallos (uno por página) y hubo **1 026**: el *fault-around* mapea 16 páginas por fallo. La hipótesis se corrigió con la medición. Se añadieron la medición en frío (`drop_caches`) y el conteo de fallos mayores.

Alucinaciones técnicas detectadas

Son propuestas que aparecieron en la exploración: respuestas de la IA al pedir alternativas para interceptar `mmap()`, tutoriales viejos de "hooking" y el propio enunciado. Ninguna se aceptó por confianza: todas se comprueban con **`verificar_ia.sh`**, que compila o busca cada cosa en los headers reales del kernel instalado. La salida está en `evidencias/verificacion_ia.txt`.

#	Sugerencia	Por qué es incorrecta	Comprobación
1	"Obtén `sys_call_table` con `kallsyms_lookup_name()` y reemplaza la entrada de `mmap`."	`kallsyms_lookup_name` **dejó de exportarse en 5.7**, y `sys_call_table` no se exporta. Además, en x86 6.9+ la tabla ya no se usa para despachar.	`grep` en `Module.symvers`: ninguno de los dos aparece.
2	"Registra tu LSM desde el módulo con `security_add_hooks()`."	Es `__init` y no se exporta; los hooks son `__ro_after_init`. Un LSM solo se registra estando compilado dentro del kernel.	No aparece en `Module.symvers`.
3	"Usa el hook LSM `file_mmap`."	Nombre de 2.6/3.x. En 6.x el hook es `mmap_file`.	`grep` en `lsm_hook_defs.h`: `file_mmap` 0 veces, `mmap_file` 1 vez.
4	"Quita el permiso con `vma->vm_flags &= ~VM_MAYWRITE`."	Desde 6.3, `vm_flags` es **const**; se modifica con `vm_flags_clear()`, que además toma el candado de la VMA.	Compilación de prueba: error *assignment of read-only member*; la versión con `vm_flags_clear` compila.
5	"Lee los argumentos del kprobe con `regs->di`, `regs->si`, `regs->dx`."	Solo existen en x86_64; en arm64 no compila.	Compilación de prueba en arm64: error *no member named 'di'*; `regs_get_kernel_argument()` compila en ambas.
6	"Desactiva la protección de escritura con `write_cr0(read_cr0() & ~0x10000)` para parchear la tabla."	**Práctica insegura:** sin desactivar interrupciones ni sincronizar con las demás CPUs abre una ventana en la que todo el kernel es escribible. Desde 5.3 el kernel **fija** el bit WP y lo restaura (`"CR0 WP bit went missing!?"`). En arm64 no existe CR0.	Compilación de prueba en arm64: no existe `write_cr0`.

#	Sugerencia	Por qué es incorrecta	Comprobación
7	"Con `maxactive` por defecto es suficiente."	Si se agotan las instancias, la llamada se ejecuta **sin revisar** (*fail-open*).	Se fijó `maxactive=256` y se reporta `nmissed` en `/proc/mmap_guard`: 0 en todas las pruebas, incluida la de estrés (800 800 llamadas concurrentes).

Problemas reales que solo aparecieron al ejecutar (no al leer el código):

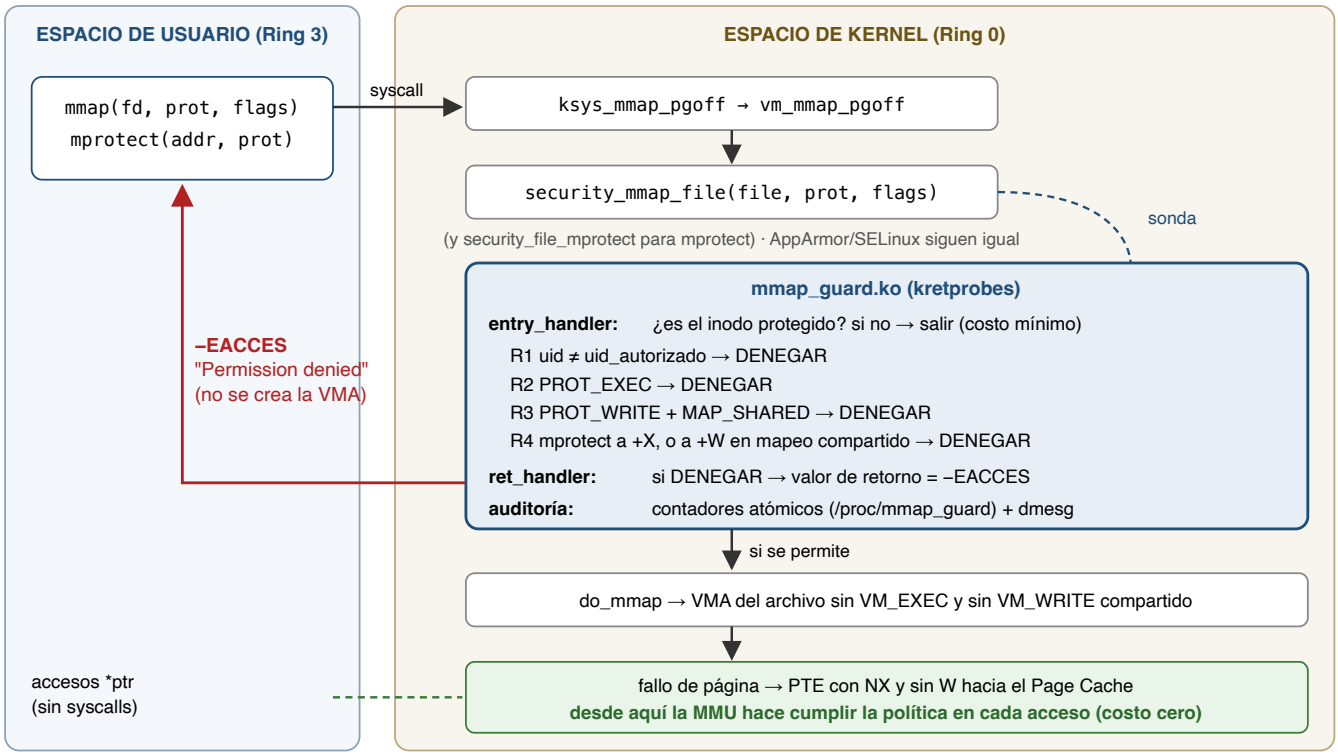
- el aviso de kbuild por el nombre del compilador;
- el mensaje engañoso de `insmod` ante `-ENOENT`;
- que en el primer arranque de la VM `apt` estuviera bloqueado y `build-essential` no se instalara (`preparar_vm.sh` ahora espera el candado y verifica al final);
- la diferencia entre fallos esperados y medidos por el *fault-around*.

Todos se corrigieron. Esto confirma la regla de trabajo: lo que la IA da por bueno hay que ejecutarlo.

Procedimiento de validación manual

1. **Compilar contra los headers de la distribución:** `make` invoca kbuild con `KDIR=/lib/modules/$(uname -r)/build` y debe terminar sin ninguna línea `warning:`. Los programas de usuario se compilan con `-Wall -Wextra`. Evidencia: `evidencias/compilacion.txt`.
2. **Verificar los símbolos:** cada función del kernel que usa el módulo debe aparecer en `Module.symvers` (`verificar_ia.sh`, sección 7), y los puntos sondeados deben existir en `/proc/kallsyms`.
3. **Probar primero la línea base:** cada ataque se corre **sin** el módulo para confirmar que el sistema es vulnerable (o que el kernel ya lo mitiga), y luego **con** el módulo. Un ataque que no funciona sin defensa no demuestra nada.
4. **Verificar la integridad fuera del proceso:** `md5sum` del archivo en disco tras cada caso, no lo que el propio PoC dice.
5. **Revisar la estabilidad:** `insmod` / `rmmod` repetidos sin *oops* en `dmesg`; `nmissed = 0`.

3.3 Arquitectura seleccionada



Por qué en `security_mmap_file` y no en el fallo de página. La decisión se toma **una vez por mapeo**, con el archivo ya resuelto y antes de que exista la VMA. Una vez negado el `VM_EXEC` o el `VM_WRITE` compartido, el **hardware** (los bits NX y R/W de la PTE) hace cumplir la política en cada acceso, gratis. Instrumentar `handle_mm_fault` cobraría en **cada** fallo de página de **todo** el sistema y no agregaría control, porque los permisos ya están fijados en la VMA.

Matriz de compensaciones (*trade-offs*)

Alternativa	Latencia de `mmap()`	Latencia de fallos	Profundidad de inspección	Cobertura	¿Bloquea?	¿Sin recompilar?	Riesgo
`LD_PRELOAD` (usuario)	≈0	0	Baja	Nula: se evade con la syscall directa	No	Sí	Bajo
Hook en `sys_call_table`	Baja	0	Media (sin archivo resuelto)	Parcial: faltan las variantes compat	Sí	Sí, pero inviable en 6.9+	**Alto** (rootkit)
LSM compilado en el kernel	Mínima	0	Alta	Total	Sí	**No**	Bajo
BPF-LSM (`lsm/mmap_file`)	Mínima	0	Alta	Total	Sí, *fail-closed*	Sí, con `lsm=bpf` al arrancar	Bajo
eBPF kprobe/fexit (solo auditoría)	Baja	0, o alta si se audita cada fallo	Alta	Total	**No**	Sí	Bajo
Instrumentar `handle_mm_fault`	0	**Alta** (todo el sistema)	Por página	Total	Sí	Sí (kprobe)	Medio
`mmap_guard` : kretprobe en `security_mmap_file` + `security_file_mprotect`	**Baja** (medida en 4.2)	**0**	**Alta** (archivo, uid, prot, flags)	**Total** (toda ruta pasa por el LSM)	**Sí**	**Sí**	Medio: *fail-open* si `nmissed > 0`

PARTE III — Implementación, presentación y argumentación

4.1 Entregables de código (`codigo/`)

Archivo	Qué es
`mmap_guard.c`	Módulo de kernel: 2 kretprobes, política R1–R4, `/proc/mmap_guard` , auditoría. Parámetros: `archivo=` y `uid_autorizado=` .
`poc_mmap.c`	7 ataques/accesos: `leer` , `escribir_compartido` , `escribir_privado` , `ejecutar` , `mprotect_escritura` , `mprotect_ejecucion` , `carrera` (Dirty COW).
`bench_mmap.c`	Mide ns por `mmap()+munmap()` y ns por fallo de página.
`Makefile`	`make` (módulo + programas, sin advertencias), `make test` , `make bench` , `make clean` .
`test_parcial3.sh`	9 casos sin el módulo y con él, `md5sum` , PASS/FAIL, auditoría.
`medir_overhead.sh`	Sobrecosto con `clock_gettime` , `strace -c` , `perf stat` .

Archivo	Qué es
`verificar_ia.sh`	Verificación de las alucinaciones contra los headers reales.
`preparar_vm.sh`	Instala headers, `strace` y `perf`; crea el usuario `atacante`; revisa Secure Boot.

Resultado de las pruebas (`evidencias/pruebas.txt`):

#	Usuario	Ataque	Sin módulo	Con módulo	Regla	Resultado
1	dueño	`mmap` lectura compartida	permitido	permitido	uso legítimo	PASS
2	atacante	`mmap` lectura compartida	permitido	**negado** (`EACCES`)	R1	PASS
3	dueño	`mmap` escritura `MAP_SHARED`	permitido, **archivo modificado**	**negado**, archivo íntegro	R3	PASS
4	dueño	`mmap` escritura `MAP_PRIVATE` (fd `O_RDONLY`)	permitido (COW), íntegro	permitido (COW), íntegro	— (pregunta 8)	PASS
5	dueño	`mmap` con `PROT_EXEC` (archivo sin bit x)	permitido	**negado**	R2	PASS
6	dueño	`mmap` lectura + `mprotect(+W)` compartido	permitido, **archivo modificado**	**negado** en `mprotect`	R4	PASS
7	dueño	`mmap` lectura + `mprotect(+X)`	permitido	**negado** en `mprotect`	R4	PASS
8	atacante	carrera Dirty COW	ataque fallido (kernel corregido)	**negado** ya en el `mmap`	R1	PASS
9	dueño	carrera Dirty COW (≈3 M `madvise`, ≈1,9 M escrituras)	ataque fallido, íntegro	ataque fallido, íntegro	— (pregunta 9)	PASS

9/9 casos correctos. Sin el módulo, **2 ataques modifican el archivo en disco** y 4 más obtienen mapeos indebidos. Con el módulo, el `md5sum` del archivo no cambia en ningún caso. Auditoría de esa corrida en `/proc/mmap_guard`: 9 `mmap` revisados (5 permitidos, 2 negados por R1, 1 por R2, 1 por R3), 2 `mprotect` negados por R4, **0 llamadas no vigiladas**.

Estabilidad (`evidencias/estabilidad.txt`):

- 20 ciclos `insmod` / `rmmod` sin errores;
- un parámetro inválido se rechaza limpio;
- estrés de 8 procesos × 100 000 `mmap` más 4 atacantes en paralelo: **800 800 llamadas revisadas, 800 negadas, 0 no vigiladas y 0 *oops*/BUG/WARNING en `dmesg`**.

4.2 Argumentación y análisis de resultados

Justificación teórica: ¿por qué en el kernel y no en espacio de usuario?

1. **No es eludible desde Ring 3.** Una biblioteca envoltorio se salta con `syscall()` directa, enlazando estático o limpiando `LD_PRELOAD`. El gancho del kernel está en la **única** ruta que lleva a crear una VMA: no hay forma de obtener un mapeo de archivo sin pasar por `security_mmap_file`.
2. **Ve la verdad, no lo que dice el usuario:** el `struct file` ya resuelto (inodo, dispositivo) y las credenciales del proceso (`current_euid()`). Renombrar el archivo o pasar un fd heredado no sirve para engañarlo.
3. **Delega en el hardware.** Al negar `VM_EXEC` y el `VM_WRITE` compartido, el kernel construye PTE con NX y sin W hacia el Page Cache. Cada acceso posterior lo controla la MMU **a velocidad de hardware y sin syscalls**, que es justo el problema que plantea el enunciado.
4. **Auditoría a prueba de manipulación:** contadores y log en memoria del kernel, que el proceso auditado no puede borrar.

Medición del sobrecosto

Las mediciones se hicieron con `medir_overhead.sh`. Cada valor es el promedio de 3 corridas; la salida completa está en `evidencias/overhead.txt`.

Medición	Sin módulo	Con módulo	Diferencia
`mmap()+munmap()` de **otro** archivo (200 000 pares, `clock_gettime`)	781,9 ns	898,3 ns	**+116 ns (+15 %)**
`mmap()+munmap()` del archivo **protegido** (se aplica la política)	777,6 ns	945,2 ns	**+168 ns (+22 %)**
Fallo de página, 64 MB **en frío** (tras `drop_caches`)	13 737 ns	13 293 ns	–3 % (ruido)
Fallo de página, 64 MB **en caché**	906,7 ns	913,0 ns	+0,7 % (ruido)
`perf stat -e page-faults` (64 MB)	1 075	1 074	igual
`perf stat` task-clock (64 MB)	1,83 ms	1,71 ms	ruido

Análisis:

1. **El costo está en `mmap()`, no en el acceso.** Unos 116 ns por llamada son el precio de la sonda: en arm64 un kprobe es una instrucción `BRK` que provoca una excepción, más el trampolín de retorno del kretprobe. Se paga en **todo** `mmap()` de archivo del sistema, aunque no sea el protegido, porque hay que mirar el inodo. Para el archivo protegido se suman unos 50 ns (reglas, contadores atómicos, chequeo del límite de log). En x86_64, los kprobes al inicio de función usan ftrace y son más baratos.
2. **Los fallos de página no cambian.** El módulo no está en esa ruta: la política ya quedó fija en `vm_flags`, y de ahí salen las PTE. Las diferencias medidas (–3 %, +0,7 %) están dentro del ruido entre corridas. Los accesos posteriores al fallo cuestan **cero**, porque los resuelve la MMU con los bits NX y R/W.
3. **Fault-around:** al tocar 16 384 páginas se producen solo 1 026 fallos, porque `filemap_map_pages` mapea 16 páginas (64 KiB) por fallo cuando ya están en el Page Cache. Esto lo confirman `getrusage` y `perf`.
4. `strace -c` confirma que el programa hace exactamente 10 006 `mmap` y 10 003 `munmap`, y que con el módulo **no aparece ninguna syscall extra** (el control ocurre dentro de `mmap`). También muestra que `strace` no sirve para medir latencia: con `ptrace` cada par sube a ≈57 µs. Por eso la latencia se midió con `clock_gettime` y `perf`.
5. **Comparación con la alternativa de instrumentar el fallo de página:** pagar ≈100 ns en cada uno de los ≈1 000 fallos por cada 64 MB de **todos** los procesos sería un costo mucho mayor. Hacerlo una vez por `mmap()` del archivo es la compensación correcta.

Limitaciones y trabajo futuro

Vector todavía abierto	Mitigación propuesta
Fail-open del kretprobe: si se agotan las `maxactive` instancias, la llamada pasa sin revisar.	Portar la política a **BPF-LSM** (`lsm/mmap_file`, `lsm/file_mprotect`), que es *fail-closed*, o a un LSM compilado. `nmissed` se vigila en `/proc`.
root puede descargar el módulo (`rmmod`) o cargar otro.	`kernel.modules_disabled=1` después de cargar; *lockdown* en modo `integrity`; Secure Boot con `CONFIG_MODULE_SIG_FORCE`.
Identidad por inodo: si el archivo se reemplaza (editor que guarda en un archivo nuevo, `rename`), el inodo cambia. Las copias no están protegidas.	Proteger el directorio, o marcar el inodo con un atributo extendido (`security.*`) y revisarlo en el gancho.
Mapeos creados antes de cargar el módulo siguen vivos.	Al cargar, recorrer los mapeos del inodo con `i_mmap` (rmap) y avisar o revocar.
`read()`/`write()` no se gobiernan: el módulo cubre la ruta de memoria.	Permisos POSIX (`chmod 600`) + `fs-verity` para integridad + `fscrypt` para confidencialidad.

Vector todavía abierto	Mitigación propuesta
`security_mmap_file` no es una ABI estable: podría renombrarse o hacerse *inline*.	`register_kretprobe` falla limpio si no existe el símbolo; BPF-LSM usa la interfaz estable.
Exploits del propio kernel (otro Dirty COW), `/dev/mem`, DMA de dispositivos.	Kernel al día; `CONFIG_STRICT_DEVMEM`; **IOMMU** contra DMA.
Ataques físicos (*cold boot*) o desde un hipervisor comprometido.	**Hardware:** cifrado de memoria (AMD SEV-SNP, Intel TDX/TME, ARM CCA). **Hipervisor:** introspección de VM (VMI) con permisos de EPT/Stage-2 que hagan de solo lectura y no ejecutables los marcos del archivo, fuera del alcance de un kernel comprometido.

Evidencias (carpeta `evidencias/`)

Archivo	Contenido
`entorno.txt`	Versión de Ubuntu, kernel, arquitectura, gcc.
`compilacion.txt`	Salida completa de `make clean && make`, sin advertencias.
`estabilidad.txt`	20 ciclos `insmod` / `rmmod`, carga con parámetro inválido, estrés de 800 800 `mmap` concurrentes, conteo de *oops* en `dmesg`.
`pruebas.txt`	Salida completa de `sudo ./test_parcial3.sh`.
`overhead.txt`	Salida completa de `sudo ./medir_overhead.sh` (incluye `strace -c` y `perf stat`).
`verificacion_ia.txt`	Salida de `./verificar_ia.sh`.
`ebpf_auditoria.txt`	Auditoría de `mmap()` con `bpftrace` (pregunta 7).